

Eng-Hin Crosslingual Retrieval Report

SemEval 2025 Task 7 — Detailed Technical Report

Cross-Track Submission

September 2026

- [1. Task Overview](#)
- [2. Data Pipeline](#)
 - [2.1 Source data](#)
 - [2.2 Parsing](#)
 - [2.3 Text construction — `build\_fused\_text`](#)
 - [2.4 Language filtering](#)
 - [2.5 Final crosslingual pair construction](#)
- [3. Retrieval Model & Fine-Tuning Method](#)
 - [3.1 Base model](#)
 - [3.2 Embedding extraction](#)
 - [3.3 Parameter-efficient fine-tuning: QLoRA](#)
 - [3.4 Contrastive training objective](#)
 - [3.5 Evaluation metric](#)
- [4. Experimental Versions — What Was Done and Why](#)
 - [4.1 Frozen baseline](#)
 - [4.2 Original fine-tune](#)
 - [4.3 v1 \(1 epoch, LR = 1e-5, LoRA rank 8\)](#)
 - [4.4 v1 + fusion \(Reciprocal Rank Fusion with frozen baseline\)](#)
 - [4.5 v2 \(LoRA rank 16, 2 epochs, quality-filtered synthetic data\)](#)
 - [4.6 v3 \(fixed dev set, reverted to v1-style settings\)](#)
 - [4.7 v3 + rerank \(cross-encoder reranking\)](#)
 - [4.8 v4 \(diverse training: synthetic English→Hindi + real English→English\)](#)
- [5. Consolidated Results Table](#)
- [6. Why Scores Go Up or Down — Summary of Root Causes](#)
- [7. Key Takeaways](#)
- [8. Caveats and Suggestions for Future Work](#)

1. Task Overview

This project addresses the **crosslingual claim retrieval** subtask of SemEval 2025 Task 7 (Multilingual and Crosslingual Fact-Checked Claim Retrieval). Given a social-media **post written in English**, the system must retrieve, from a large pool of **fact-checked claims written in Hindi**, the top-K claims that the post is actually about. Success is measured only against a fixed test split with a released ground-truth mapping (`crosslingual_reference.json`).

This is a **dense bi-encoder retrieval** problem: posts and claims are embedded into the same vector space, and retrieval is done by cosine similarity (nearest-neighbour search), not by generative matching.

Why this is hard:

- The query (English post, often noisy social-media text, sometimes only recoverable via OCR of an attached image) and the target (a formally-worded Hindi fact-check claim) are in **different languages and different registers**.

- There is no large parallel corpus of English-post ↔ Hindi-claim pairs to train on directly — such pairs had to be **synthesized** (via machine translation of the English side, or vice versa) before contrastive training was even possible.
- The evaluation pool of Hindi claims is large, so the retriever must generalize, not memorize.

2. Data Pipeline

2.1 Source data

Four SemEval-provided files, split into a `train_dev_sets` and `test_set` directory:

File	Contents
<code>fact_checks.csv</code>	All fact-check claims, each <code>claim</code> field stored as a stringified tuple <code>(original_text, translated_text, lang_probs)</code>
<code>posts.csv</code>	All social-media posts, <code>text</code> and <code>ocr</code> fields in the same stringified-tuple format
<code>pairs.csv</code> / <code>pairs_test_crosslingual.csv</code>	Known (<code>post_id</code> , <code>fact_check_id</code>) links
<code>tasks.json</code> / <code>crosslingual_reference.json</code>	Task-specific ID pools and the ground-truth post→claim mapping used for scoring

2.2 Parsing

Because `claim` and `text / ocr` columns are Python-literal-encoded tuples embedded in CSV cells, `ast.literal_eval` (wrapped in `safe_parse_tuple`) is used to safely deserialize each cell, defaulting to `None` on malformed input rather than raising.

2.3 Text construction — `build_fused_text`

For each post, the **main text** and **OCR text** (text extracted from any attached image) are combined:

- If OCR text is a substring of the main text (case-insensitively), it is **dropped** — this avoids duplicating content that a human transcribed and an OCR engine re-extracted from the same image.
- If only one of the two exists, that one is used.
- Otherwise they are concatenated as `"{main_text} [OCR]: {ocr_text}"`.

This is a sensible heuristic: OCR text is frequently noisy, and blindly concatenating it with clean caption text would inject redundant or garbled tokens into the embedding input.

2.4 Language filtering

English posts are identified via the third element of the parsed tuple, a list of `(lang_code, probability)` pairs (or a single `(lang_code, prob)` pair). The dominant language is taken as the highest-probability code, and a post is kept if that code is `eng` or `en`.

Hindi claims are identified by **either** of two independent signals (logical OR): 1. A **Devanagari-script regex** (`([\u0900-\u097F])`) match on the claim's original text. 2. The claim's dominant language code being `hin` or `hi`.

Using both signals is a deliberate robustness choice — some claims may be tagged with a language code but written in Romanized Hindi (missed by the regex), while others may lack a reliable language tag but be unambiguously in Devanagari script (missed by the code check alone). The OR combination catches both cases

at the cost of a small risk of false positives (e.g., a claim with a single stray Devanagari character misclassified as Hindi) — an acceptable trade-off for this filtering step.

2.5 Final crosslingual pair construction

The `pairs_test_crosslingual.csv` records are restricted to `(post_id, fact_check_id)` rows where the post is in the English-detected ID set **and** the fact-check is in the Hindi-detected ID set, then de-duplicated. This produces the exact test query/target universe scored against `crosslingual_reference.json`.

3. Retrieval Model & Fine-Tuning Method

3.1 Base model

`intfloat/multilingual-e5-large` — a multilingual sentence-embedding model pretrained with the E5 contrastive recipe, which requires **task prefixes** on its inputs (`"query: "` for the retrieval query, `"passage: "` for the candidate document). Both prefixes are applied correctly and consistently throughout the notebook.

3.2 Embedding extraction

- **Mean pooling** over the last hidden state, masked by the attention mask (not just naive averaging — padded positions are correctly excluded via `attention_mask`).
- **L2 normalization** of the pooled embedding, so that a dot product between two embeddings is equivalent to cosine similarity.

3.3 Parameter-efficient fine-tuning: QLoRA

Rather than fine-tuning all ~560M parameters of the base model, the notebook uses **QLoRA**:

- The base model is loaded in **4-bit NF4 quantization** (`BitsAndBytesConfig`, double quantization, `fp16` compute dtype) to fit training on a single consumer/Colab GPU.
- **LoRA adapters** are injected into the attention projection layers (`query`, `key`, `value`, and in later variants `dense`), with rank `r` and scaling `lora_alpha` varied across experiments (see §4).
- Only the small LoRA adapter weights are trained; the quantized base weights stay frozen.

3.4 Contrastive training objective

Training examples are triplets: `(query, positive, [negatives])`.

- Each batch encodes the queries, positives, and a flattened set of **hard negatives** (3 per example, sampled from a larger negative pool per example) through the model.
- A similarity matrix is formed: `logits = query_embeddings @ [positive_embeddings; negative_embeddings]^T / temperature` (temperature = 0.05).
- The loss is **in-batch + hard-negative InfoNCE** (softmax cross-entropy where the correct positive index is the label) — a standard and effective contrastive retrieval loss. In-batch negatives (other examples' positives, incidentally acting as negatives for a given query) supplement the explicit hard negatives, giving a richer negative signal per step than hard negatives alone.

3.5 Evaluation metric

Success@K (also called Recall@K / Hit-Rate@K in some literature): for each query, check whether **any** of the true positive fact-check IDs appear within the top-K retrieved candidates; the reported score is the fraction of

queries where this holds, computed for $K = 1, 5, 10$. This matches the SemEval task's official metric.

4. Experimental Versions — What Was Done and Why

Eight configurations were evaluated on the real English→Hindi test set. All numbers below are exactly as recorded from the notebook's evaluation cells.

4.1 Frozen baseline

The unmodified `multilingual-e5-large` model (fp16, no LoRA, no fine-tuning) is used directly for retrieval. This establishes the reference point: **how good is off-the-shelf multilingual embedding at this task with zero task-specific adaptation.**

Result: Success@1 = 0.1812, @5 = 0.5125, @10 = 0.6375

This is a surprisingly strong baseline — the model already places semantically related English and Hindi text reasonably close in embedding space, purely from its multilingual pretraining. This baseline turned out to be a difficult bar to beat, which shapes the rest of the story.

4.2 Original fine-tune

The first fine-tuning attempt: LoRA adapters trained on synthetic English→Hindi pairs (English post paired with a **machine-translated** Hindi version of the corresponding English claim, since no natural English-post/Hindi-claim training pairs exist).

Result: Success@1 = 0.0375, @5 = 0.2000, @10 = 0.2812 — a catastrophic collapse relative to the frozen baseline (roughly a 5x drop at $K=1$, and more than 2x at $K=10$).

Why this happened (diagnosis):

1. **Overfitting to synthetic-translation artifacts.** The Hindi “positives” used in training were machine-translated, not natural Hindi. Machine translation output has systematic statistical fingerprints (particular phrasing, sentence structure, transliteration choices) that differ from naturally-authored Hindi fact-checks. The model likely learned to align English text with *translationese* Hindi rather than with the semantic content itself — a shortcut that fails at test time because the actual test-set Hindi claims are naturally written, not translated.
2. **No held-out validation signal to catch it early.** This run had no properly isolated dev set distinct from the training distribution, so there was no early warning that the model was diverging from genuine cross-lingual semantic alignment.
3. **Full-strength training without capacity/step control.** Without the more conservative learning rate and epoch budget used in later versions, the small set of LoRA parameters had enough gradient steps to move the embedding space substantially away from its well-pretrained starting point — a classic case of catastrophic forgetting of general-purpose multilingual alignment in exchange for narrow overfitting to the synthetic training distribution.

This result was the pivotal turning point of the project: it demonstrated that **naive fine-tuning on synthetic cross-lingual pairs actively hurts** the task, and every subsequent version was, in effect, an attempt to fix this failure mode.

4.3 v1 (1 epoch, LR = 1e-5, LoRA rank 8)

Response to the collapse: **much lower learning rate** (1e-5 vs. whatever higher rate the original attempt used) and **only 1 epoch** instead of a longer schedule, with a conservative LoRA configuration (`r=8`, `alpha=16`, standard `query/key/value` target modules).

Result: Success@1 = 0.2062, @5 = 0.4875, @10 = 0.6188

- **Success@1 improved past the frozen baseline** (0.2062 vs 0.1812) — the model gained *some* genuine task-specific signal.
- **Success@5 and @10 slightly regressed** relative to frozen (0.4875 vs 0.5125; 0.6188 vs 0.6375).

Why the mixed result: a short, low-LR fine-tune is gentle enough to avoid the full collapse seen before, and it does sharpen the model's top-1 precision on the specific claim phrasing patterns seen in synthetic training. But it still nudges the embedding space slightly away from the frozen model's broader, more robust multilingual alignment — enough to lose a bit of recall further down the ranked list (K=5, K=10), even while gaining at the very top (K=1). This is a textbook **precision/generalization trade-off** from partial overfitting: better at the easiest, most-in-distribution queries, slightly worse at the harder tail.

4.4 v1 + fusion (Reciprocal Rank Fusion with frozen baseline)

Rather than trusting the fine-tuned model alone, its ranked list is combined with the frozen baseline's ranked list using **Reciprocal Rank Fusion (RRF)**:

```
score(candidate) =  $\sum$  over each ranked list [  $1 / (k + \text{rank\_in\_that\_list} + 1)$  ]
```

with `k = 60`. Candidates are then re-sorted by this fused score. RRF is a simple, calibration-free way to combine two rankers without needing their raw similarity scores to be on comparable scales.

Result: Success@1 = 0.1938, @5 = 0.5000, @10 = 0.6375

- Success@1 fell slightly *below* v1 alone (0.1938 vs 0.2062) — fusing in the frozen model's ranking pulled some of v1's confident-but-fine-tuned top-1 picks down.
- Success@5 and @10 **recovered** almost exactly to frozen-baseline levels (0.5000→0.5125 gap closes to a rounding difference; @10 matches frozen exactly at 0.6375).

Why: fusion acts as an ensembling safety net — it cannot do better than the best individual ranker at every single query, but it smooths over cases where one model is wrong and the other is right, recovering most of the frozen model's broad recall while still injecting a little of the fine-tuned model's task-specific signal. It is a “do no harm” strategy rather than a “clear improvement” strategy at this stage.

4.5 v2 (LoRA rank 16, 2 epochs, quality-filtered synthetic data)

An attempt to *do more* than v1: increase adapter capacity (`r=16`, `alpha=32` , wider target modules including `dense`), train for 2 epochs instead of 1, and **filter the synthetic training positives by Devanagari-character ratio** (`devanagari_ratio > 0.3`) to remove translation outputs that were low-quality or fell back to English/mixed script.

Result: Success@1 = 0.1500, @5 = 0.4250, @10 = 0.5437 — the **worst fine-tuned result** after the original collapse, regressing below both v1 and the frozen baseline on every metric.

Why this got worse, not better:

1. **More capacity + more epochs re-opened the overfitting risk** that v1's conservative settings had specifically avoided. Doubling the epoch count and widening LoRA's target modules (including the `dense` layer) gives the adapter more room to specialize to the synthetic training distribution's quirks, at the direct expense of the frozen model's general-purpose alignment — the same failure mode as the original collapse, just less severe because the LR/config wasn't as aggressive.
2. **The quality filter likely reduced effective data diversity.** Keeping only high-Devanagari-ratio translations discards a portion of the training pairs, which — combined with more epochs on a now-smaller, more homogeneous set — increases the chance of the model memorizing surface patterns in what remains rather than learning transferable alignment.
3. This result is direct evidence that **“try harder” (more capacity, more epochs, cleaner-looking data) is not automatically better** when the underlying training signal (synthetic translations) has a

systematic distribution mismatch with the real evaluation distribution (natural Hindi). Fixing the *symptom* (translation noise) without fixing the *root cause* (synthetic vs. natural language mismatch) made the overfitting problem worse, not better.

4.6 v3 (fixed dev set, reverted to v1-style settings)

A **methodological correction** rather than a modeling change: LoRA settings reverted to v1's conservative configuration (`r=8`, `alpha=16`, standard target modules, 1 epoch, `LR=1e-5`), but this time evaluated during training against a **properly constructed, leak-free "fixed dev set"** — held-out synthetic positives combined with real natural-language crosslingual claims as distractors, so that checkpoint selection is no longer contaminated by overlap with the training distribution the way earlier ad-hoc dev checks may have been.

Result: Success@1 = 0.1938, @5 = 0.5062, @10 = 0.6375

This is nearly identical to v1 (0.2062/0.4875/0.6188) and v1+fusion (0.1938/0.5000/0.6375) — small differences here are consistent with ordinary run-to-run variance (no `torch.manual_seed()` was set for the training loop itself, only for negative sampling and data splits, so batch order and initialization were not fully controlled between runs). The real value of v3 is not a score jump; it is a **methodologically sounder checkpoint-selection process** that the later v4 experiment builds on with more confidence.

4.7 v3 + rerank (cross-encoder reranking)

The top-10 candidates retrieved by v3's bi-encoder are **reranked** using a separate, more expensive **cross-encoder** model (`cross-encoder/mmarco-mMiniLMv2-L12-H384-v1`, trained on mMARCO, which includes Hindi), which jointly encodes the (query, candidate) pair and outputs a direct relevance score — a strictly higher-fidelity but much slower comparison than independent bi-encoder embeddings.

Result: Success@1 = 0.1812, @5 = 0.5188, @10 = 0.6375

- Success@1 actually **dropped slightly** relative to v3 (0.1812 vs 0.1938).
- Success@5 **improved** (0.5188 vs 0.5062).
- Success@10 is **unchanged** (0.6375 vs 0.6375) — this is mechanically expected, since reranking only reorders the same top-10 set the bi-encoder already retrieved; it cannot pull in a correct answer that wasn't in the top-10 to begin with, so Success@10 is invariant to reranking by construction.

Why the mixed result: the cross-encoder was trained on general multilingual passage-ranking data (mMARCO), not on this task's specific fact-check/social-media-post register, so it has its own biases about what looks "relevant" that don't perfectly match this dataset's notion of a matching claim. It correctly promotes some true positives from rank 2-5 into the top-5 (helping Success@5), but occasionally also demotes the bi-encoder's correct rank-1 pick in favor of a candidate that looks superficially more lexically similar (hurting Success@1). Net effect: a wash, with the benefit concentrated in the middle of the ranking rather than at the very top.

4.8 v4 (diverse training: synthetic English→Hindi + real English→English)

The final and best-performing configuration. The key change: the training set is no longer purely synthetic English→Hindi pairs. It is **augmented with real, natural (non-translated) English post ↔ English claim pairs** drawn from the monolingual English task data, combined and shuffled together with the synthetic crosslingual pairs before training (same conservative `r=8`-equivalent capacity and 1-epoch, low-LR regime as v3, applied to this combined dataset).

Result: Success@1 = 0.2625, @5 = 0.5813, @10 = 0.6500 — the **only version that beats the frozen baseline on all three metrics simultaneously**, and by a clear margin at K=1 and K=5.

Why this worked where everything else struggled:

1. **Real pairs anchor the model to genuine semantic alignment.** The English-English pairs are entirely

natural language on both sides — no translation artifacts to overfit to. Training on a mix forces the LoRA adapter to learn a query-to-passage alignment signal that must work for *both* the clean, natural pairs *and* the noisier synthetic crosslingual pairs simultaneously, which regularizes the adapter away from latching onto translation-specific shortcuts (the exact failure mode diagnosed in §4.2).

2. **Effectively a form of data augmentation / multi-task regularization.** Rather than fixing the synthetic data’s translation-artifact problem directly (as v2’s quality filter attempted, unsuccessfully), v4 dilutes its influence by mixing in a data source with a fundamentally different, complementary bias — a standard and often very effective technique when one data source is scarce or noisy but a related, cleaner source is available.
3. **It still respects the “don’t overtrain” lesson from v1/v2.** Capacity and epoch budget were kept conservative (matching v3, not v2’s larger/longer configuration), so the gains here come specifically from *what* the model was trained on, not from training it harder.

5. Consolidated Results Table

Version	Success@1	Success@5	Success@10	Δ @1 vs. baseline	Δ @5 vs. baseline	Δ @10 vs. baseline
Frozen baseline	0.1812	0.5125	0.6375	—	—	—
Original fine-tune	0.0375	0.2000	0.2812	−79.3%	−61.0%	−55.9%
v1 (1ep, LR1e-5)	0.2062	0.4875	0.6188	+13.8%	−4.9%	−2.9%
v1 + fusion	0.1938	0.5000	0.6375	+6.9%	−2.4%	0.0%
v2 (rank16, 2ep)	0.1500	0.4250	0.5437	−17.2%	−17.1%	−14.7%
v3 (fixed dev)	0.1938	0.5062	0.6375	+6.9%	−1.2%	0.0%
v3 + rerank	0.1812	0.5188	0.6375	0.0%	+1.2%	0.0%
v4 (diverse train)	0.2625	0.5813	0.6500	+44.9%	+13.4%	+2.0%

(Percentages are relative change vs. the Frozen Baseline row, i.e. $(\text{value} - \text{baseline}) / \text{baseline} \times 100$.)

6. Why Scores Go Up or Down — Summary of Root Causes

Direction	Observed in	Root cause
Large decrease	Original fine-tune	Training exclusively on machine-translated synthetic positives teaches the model translation-artifact shortcuts instead of genuine cross-lingual semantics; no held-out signal caught the drift early; catastrophic forgetting of the base model’s pretrained multilingual alignment.
Moderate decrease	v2 (rank16, 2ep)	Increasing adapter capacity and training length re-opens the same overfitting risk that caused the original collapse, even with a quality filter on

		the data — more capacity on a still-synthetic, still-translation-biased dataset amplifies memorization rather than generalization.
Small decrease	v1, v3 @5/@10	A short, low-LR fine-tune shifts the embedding space slightly away from the frozen model's broad, well-calibrated multilingual alignment; sharpens top-1 accuracy on in-distribution patterns at a small cost to deeper recall.
Small decrease then flat	v3+rerank @1	Cross-encoder reranking, trained on out-of-domain data (mMARCO), occasionally displaces the bi-encoder's correct top-1 pick in favor of a lexically-similar but incorrect candidate; @10 is mechanically unaffected since reranking only reorders an already-fixed candidate set.
Small increase	v1 @1, v3 @5, v3+rerank @5	Mild task-specific fine-tuning or reranking correctly sharpens ranking within the top-K for queries close to the training/reranker's distribution, without enough drift to hurt the harder cases.
Recovery to baseline	v1+fusion	Reciprocal Rank Fusion ensembles the fine-tuned and frozen rankings, smoothing out the fine-tuned model's localized errors and restoring most of the frozen baseline's broader recall.
Large, consistent increase	v4 (diverse train)	Mixing real natural-language English-English pairs with synthetic English-Hindi pairs regularizes the adapter against translation-artifact overfitting while still injecting genuine cross-lingual signal, all under the same conservative capacity/epoch budget that avoided collapse in v1/v3.

7. Key Takeaways

1. **A strong pretrained multilingual embedding model is a genuinely hard baseline to beat.** `multilingual-e5-large`, used with zero task-specific fine-tuning, already achieves $\text{Success@10} = 0.6375$ — a bar that 5 of the 7 fine-tuning attempts failed to reach or only matched.
2. **Synthetic (machine-translated) training data is a double-edged sword.** It is often the only way to bootstrap training pairs for a low-resource language direction, but training on it *directly and exclusively* risks teaching the model to exploit translation artifacts rather than real semantics — this was the single biggest failure mode in the whole project (§4.2).
3. **“More capacity, more training” is not a reliable fix when the data itself is biased.** v2 demonstrated that scaling up LoRA rank and epochs on the same flawed synthetic-only data makes overfitting worse, not better.
4. **Mixing in a small amount of clean, real, related data is more effective than more of the same biased data.** v4's combination of real English-English pairs with synthetic English-Hindi pairs was the single change that turned a struggling fine-tuning effort into a genuine, consistent improvement over the

frozen baseline.

5. **Ensembling (fusion) and reranking are useful safety nets, not silver bullets.** Both v1+fusion and v3+rerank recovered lost performance or nudged individual metrics slightly upward, but neither produced the kind of across-the-board gain that a better training *data mixture* (v4) did.
 6. **Final recommended model: v4**, with Success@1 = 0.2625, Success@5 = 0.5813, Success@10 = 0.6500 — the only configuration that improves on the frozen baseline at every reported cutoff.
-

8. Caveats and Suggestions for Future Work

- **Frozen baseline (fp16) vs. fine-tuned models (4-bit NF4 quantized)** are not evaluated under identical numerical precision. Part of the gap between them could be attributable to quantization noise rather than fine-tuning alone; evaluating the frozen model under the same 4-bit quantization would isolate the LoRA adapter’s true marginal contribution.
- **Training-loop randomness was not fully seeded** (`torch.manual_seed()` was not set; DataLoader shuffling was unseeded), so small score differences between very similar configurations (e.g., v1 vs. v3) may partly reflect run-to-run variance rather than the specific methodological change being tested.
- **Dev-set metrics used for checkpoint selection during training differ from the final reported Success@K on the real test set** — a reasonable measure to avoid test-set leakage, but it means the “best” checkpoint by training-time dev score is not guaranteed to be the best by final test score.
- **The synthetic translation and hard-negative mining pipeline itself (the source of `train_examples_eng_hin_synth.pkl`, `claim_translations_eng_to_hin.json`) was produced upstream of this notebook** and is not documented here; a full reproducibility write-up would need to specify the translation model/method and negative-mining strategy used to build those files.